

Evaluación Técnica para Desarrollador Backend (Python)

Descripción General

Esta evaluación técnica está diseñada para evaluar habilidades en desarrollo backend utilizando Python. Se requiere construir una API RESTful simple para un sistema de procesamiento de mensajes de chat. Se busca evaluar las capacidades para:

1. Diseñar e implementar una API limpia y mantenible
2. Trabajar con Python y frameworks backend comunes
3. Implementar un manejo de errores y validación adecuados
4. Escribir pruebas unitarias
5. Documentar código y API

Descripción del Proyecto

Crear una API simple de procesamiento de mensajes que:

1. Recibe mensajes de chat
2. Valida el formato del mensaje
3. Procesa el mensaje (aplica transformaciones simples)
4. Almacena el mensaje en una base de datos
5. Devuelve respuestas apropiadas

Requisitos

Stack Tecnológico

- Python 3.10+
- FastAPI o Flask (elige uno)
- SQLite para la base de datos (por simplicidad)
- Pytest para pruebas

Requisitos Funcionales

1. Endpoint de Mensajes

- Crear un endpoint POST `/api/messages` que acepte cargas JSON
- El endpoint debe validar los mensajes entrantes
- Procesar mensajes válidos y almacenarlos en la base de datos
- Devolver códigos de estado HTTP y respuestas apropiadas

2. Esquema de Mensaje

- Cada mensaje debe tener:
 - ◆ `message_id`: Identificador único (string)
 - ◆ `session_id`: Identificador de sesión (string)
 - ◆ `content`: Contenido del mensaje (string)
 - ◆ `timestamp`: Marca de tiempo del mensaje (formato ISO datetime)
 - ◆ `sender`: Quién envió el mensaje (string, "user" o "system")

3. Procesamiento de Mensajes

- Implementar un pipeline de procesamiento simple que:
 - ◆ Valide el formato del mensaje
 - ◆ Verifique y filtre contenido inapropiado (un filtrado simple de palabras es suficiente)
 - ◆ Agregue metadatos (ej., longitud del mensaje, conteo de palabras)

4. Recuperación de Mensajes

- Crear un endpoint GET `/api/messages/{session_id}` que devuelva todos los mensajes para una sesión dada
- Incluir soporte para paginación (limit/offset)
- Permitir filtrado por remitente

5. Manejo de Errores

- Implementar un manejo de errores adecuado para:
 - ◆ Formato de mensaje inválido
 - ◆ Campos requeridos faltantes
 - ◆ Errores del servidor
- Devolver códigos de estado HTTP y mensajes de error apropiados

Organización del Código

Organiza el código siguiendo los principios de arquitectura limpia:

- Separar responsabilidades (controladores, servicios, repositorios)
- Usar inyección de dependencias donde sea apropiado
- Seguir principios SOLID

Entregables

1. Código Fuente

- Proyecto Python completo con todos los archivos requeridos
- Archivo de requisitos (requirements.txt)
- README con instrucciones de configuración y uso

2. Pruebas

- Pruebas unitarias para todos los componentes
- Pruebas de integración para los endpoints de la API
- Al menos 80% de cobertura de pruebas

3. Documentación

- Documentación de la API (puede estar en el README o en un archivo separado)
- Comentarios en el código donde sea necesario
- Instrucciones de configuración

Criterios de Evaluación

1. **Funcionalidad:** ¿La API funciona según lo especificado?
2. **Calidad del Código:** ¿El código está bien organizado, es legible y mantenible?
3. **Pruebas:** ¿Hay pruebas completas? ¿Pasan?
4. **Manejo de Errores:** ¿El manejo de errores es robusto y amigable para el usuario?
5. **Documentación:** ¿La API y el código están bien documentados?

Puntos Extra (Opcional)

- Implementar un mecanismo de autenticación simple
- Agregar un endpoint WebSocket para actualizaciones de mensajes en tiempo real
- Implementar funcionalidad de búsqueda de mensajes

- Agregar soporte Docker para fácil despliegue
- Implementar un mecanismo simple de limitación de tasa
- Proponer infraestructura con IaC

Instrucciones de Entrega

1. Crear un nuevo repositorio GitHub para el proyecto
2. Hacer commit de tu código al repositorio
3. Incluir un README con:
 - Descripción general del proyecto
 - Instrucciones de configuración
 - Documentación de la API
 - Instrucciones para pruebas
4. Compartir el enlace del repositorio cuando esté completo

¡Buena suerte!

Ejemplo de Carga de Mensaje

```
{
  "message_id": "msg-123456",
  "session_id": "session-abcdef",
  "content": "Hola, ¿cómo puedo ayudarte hoy?",
  "timestamp": "2023-06-15T14:30:00Z",
  "sender": "system"
}
```

Ejemplo de Respuesta

```
{
  "status": "success",
  "data": {
    "message_id": "msg-123456",
    "session_id": "session-abcdef",
  }
}
```

```
{
  "content": "Hola, ¿cómo puedo ayudarte hoy?",
  "timestamp": "2023-06-15T14:30:00Z",
  "sender": "system",
  "metadata": {
    "word_count": 6,
    "character_count": 32,
    "processed_at": "2023-06-15T14:30:01Z"
  }
}
```

Ejemplo de Respuesta de Error

```
{
  "status": "error",
  "error": {
    "code": "INVALID_FORMAT",
    "message": "Formato de mensaje inválido",
    "details": "El campo 'sender' debe ser 'user' o 'system'"
  }
}
```